Aerospace Predictive Maintenance

Version 2 — scikit-learn Production Architecture

This document describes **src_v2/**, now rebuilt on real scikit-learn ensemble algorithms (IsolationForest for anomaly detection, RandomForestRegressor for Remaining Useful Life) instead of the from-scratch numpy implementations used in the first pass at v2. It supersedes the previous code_documentation_v2.pdf, which described that from-scratch version; that version's files have been removed from the project so only this scikit-learn implementation remains.

v1 is unaffected. The original src/, README.md, and code_documentation.pdf still describe the original hackathon MVP exactly as before.

■ ***Important note on verification:*** *the sandbox this document was generated in has no outbound internet access and does not have scikit-learn installed, so this code could be syntax-checked (python -m py_compile, all files pass) but NOT executed or benchmarked end-to-end the way earlier versions were. Please run train.py and predict_service.py on your machine (where scikit-learn is installed) to confirm behavior and get real benchmark numbers before relying on this for a live demo.*

What Changed From the From-Scratch v2

Aspect	Value	Notes
Anomaly detection engine	sklearn.ensemble.IsolationForest	Replaces the custom numpy _IsolationTree/IsolationForest classes (deleted)
RUL estimation engine	sklearn.ensemble.RandomForestRegressor	Replaces the custom numpy _DecisionTreeRegressor/RandomForestRegressor classes (deleted)
Ensemble size	IsolationForest: 200 trees. Random Forest: 300 trees, max_depth=12	Raised from the from-scratch version's 100/25 - affordable now that fitting runs in optimized C rather than a Python loop
Model persistence	joblib.dump / joblib.load (.joblib files)	Replaces raw pickle (.pkl) - the scikit-learn-recommended approach for numpy-heavy estimators
Feature importances	model.feature_importances_ (built-in attribute)	Replaces the custom split-count proxy in the from-scratch Random Forest
Files removed	models/isolation_forest.py, models/random_forest.py, old *.pkl artifacts	Deleted outright per instructions to keep only the updated version
Files unchanged	data_simulator_v2.py, features.py, evaluation_v2.py, dashboard_v2.py, mro_integration_v2.py	These never depended on which ML library trains the models

File-by-File Documentation

config.py

Central configuration. Hyperparameters now target scikit-learn's constructors directly (IFOREST_N_ESTIMATORS, IFOREST_MAX_SAMPLES, IFOREST_MAX_FEATURES, IFOREST_N_JOBS; RF_N_ESTIMATORS, RF_MAX_DEPTH, RF_MIN_SAMPLES_SPLIT, RF_MIN_SAMPLES_LEAF, RF_MAX_FEATURES, RF_N_JOBS). Ensemble sizes were raised (200 / 300 trees) since scikit-learn's compiled, parallelized (n_jobs=-1) implementation makes larger ensembles cheap where the from-scratch pure-Python version could not afford them.

data_simulator_v2.py, features.py (unchanged)

Identical to the previous v2 pass: a 40-aircraft, 160-part, 5-sensor synthetic fleet with an explicit train/validation/test split by unit, and 31 engineered features (raw reading + rolling mean/std/min/max/slope per sensor, plus cycle number). These modules have no dependency on which ML library trains the downstream models, so they were left as-is.

anomaly_detection_v2.py

Now trains a real sklearn.ensemble.IsolationForest per component instead of the from-scratch version. The overall architecture is unchanged: train on the TRAIN split, tune each component's alert threshold on the VALIDATION split, then score any batch of data.

Function	Inputs	Output	What it does
`_anomaly_score(model, X)`	model: fitted IsolationForest X: 2D numpy array	1D numpy array: anomaly score per row	scikit-learn's model.score_samples(X) returns LOWER values for more abnormal points (opposite of the original paper's convention); this negates it so higher = more anomalous everywhere else in the project.
`train_anomaly_models( featured_train_df)`	featured_train_df: engineered features, TRAIN split only	dict {component: fitted IsolationForest}	Fits sklearn.ensemble.IsolationForest(n_estimators=200, max_samples=256, contamination=0.05, n_jobs=-1) separately per component type.
`compute_thresholds( models, featured_val_df, contamination=0.05)`	models: dict from train_anomaly_models featured_val_df: VALIDATION split contamination: float	dict {component: threshold score}	Scores the validation split with _anomaly_score and picks each component's threshold as the matching score quantile - a data-driven cutoff, deliberately independent of sklearn's own internal contamination-based cutoff.
`score_anomalies( models, thresholds, featured_df)`	models, thresholds: from the two functions above featured_df: any engineered-feature DataFrame	same DataFrame + anomaly_score, anomaly_threshold, is_anomaly columns	Scores every row with its component's model and flags it if the score exceeds that component's tuned threshold.

rul_model_v2.py

Now trains a real sklearn.ensemble.RandomForestRegressor per component instead of the from-scratch version.

Function	Inputs	Output	What it does
`train_rul_models( featured_train_df)`	featured_train_df: TRAIN split	dict {component: fitted RandomForestRegressor}	Fits sklearn.ensemble.RandomForestRegressor(n_estimators=300, max_depth=12, min_samples_split=10, min_samples_leaf=4, max_features='sqrt', n_jobs=-1) separately per component type.
`predict_rul(models, df)`	models: dict from train_rul_models df: engineered-feature DataFrame	1D numpy array: predicted RUL per row (clipped at 0)	Routes each row to the right component's model.
`get_feature_importances( models)`	models: dict from train_rul_models	dict {component: {feature_name: importance}}	Reads scikit-learn's built-in model.feature_importances_ attribute for each component's model and pairs it with the feature names - lets an engineer see which sensors/statistics actually drive each component's RUL prediction.
`evaluate(y_true, y_pred)`	y_true, y_pred: numpy arrays	dict {'MAE', 'RMSE'}	Standard regression accuracy metrics (unchanged).
`evaluate_by_component( df_with_predictions)`	df_with_predictions: DataFrame with RUL + predicted_RUL	dict {component: {'MAE','RMSE','n_rows'}}	Per-component accuracy breakdown (unchanged).

model_registry.py

Model persistence now uses joblib (joblib.dump / joblib.load) instead of raw pickle - the scikit-learn-recommended serialization approach, generally more efficient for estimators holding large numpy arrays (like a 300-tree forest). Files are now named _.joblib instead of .pkl. The model card JSON gained a library field ('scikit-learn') and an sklearn_estimator field recording the exact estimator class name, so anyone inspecting artifacts/model_card.json can see at a glance what produced each file.

Function	Inputs	Output	What it does
`save_models( models_by_component, name, artifact_dir, hyperparameters, metrics, extra)`	models_by_component: dict {component: fitted sklearn estimator} name: str artifact_dir: str hyperparameters, metrics, extra: dict	None (writes .joblib files + updates model_card.json)	Saves each component's estimator with joblib.dump and records a model-card entry including which scikit-learn class was used.
`load_models( name, components, artifact_dir)`	name: str components: list of str artifact_dir: str	dict {component: unpickled/loaded sklearn estimator}	Loads back exactly what save_models wrote, via joblib.load.
`load_model_card( artifact_dir)`	artifact_dir: str	dict (empty if no card exists yet)	Reads the JSON model card (unchanged behavior).

dashboard_v2.py, mro_integration_v2.py, evaluation_v2.py (unchanged)

These modules only consume the predicted_RUL / anomaly_score / is_anomaly columns that anomaly_detection_v2.py and rul_model_v2.py attach - they have no idea whether those columns came from a from-scratch model or scikit-learn, so nothing here needed to change. This is exactly the payoff of keeping a stable function-based interface between pipeline stages.

train.py / predict_service.py (train/serve split, mostly unchanged)

The overall offline-training / online-serving architecture is unchanged. train.py's saved hyperparameter dictionary now records n_estimators instead of n_trees to match scikit-learn's constructor argument names. predict_service.py required no changes at all - it only calls the wrapper functions (score_anomalies, predict_rul, load_models), which now happen to be backed by scikit-learn instead of custom numpy code.

tests/test_models.py

Rewritten to test the scikit-learn-backed wrapper functions rather than internal tree-building logic (which no longer exists in this project). Feature-engineering tests (_slope, add_rolling_features, feature_column_names) are unchanged and require no ML library. The anomaly-detection and RUL tests build a small synthetic 'toy' featured dataset, temporarily substitute a smaller feature list (matching what the toy data actually contains) via monkeypatching, then check that: the anomaly detector's is_anomaly output is always 0 or 1, and the Random Forest's predictions beat a naive 'always predict the training mean' baseline on held-out cycles. These tests require scikit-learn to be installed to run (python -m unittest tests.test_models -v from src_v2/) - see the verification note on the cover page.

Why scikit-learn Instead of the From-Scratch Implementation

- **Battle-tested, optimized implementation:** scikit-learn's IsolationForest and RandomForestRegressor are implemented in Cython/C, extensively tested across the ML community, and handle edge cases (missing-value behavior, degenerate splits, numerical stability) more robustly than a from-scratch version built in a time-boxed sandbox exercise.
- **Speed at real ensemble sizes:** the from-scratch version had to cap itself at 25-100 trees and shallow depths to finish training in under a minute in pure Python. scikit-learn's compiled, parallelized (n_jobs=-1) trees make 200-300-tree ensembles with deeper trees (max_depth=12 for the Random Forest) practical, which generally improves both accuracy and stability.
- **Ecosystem compatibility:** a scikit-learn Pipeline/estimator is immediately compatible with the broader MLOps tooling ecosystem - joblib persistence, sklearn's model inspection utilities (feature_importances_, partial dependence, permutation importance), and any future hyperparameter search (GridSearchCV/RandomizedSearchCV) - all work out of the box, whereas a custom implementation would need each of those rebuilt by hand.
- **Maintainability:** the custom tree code, while correct (it passed its own unit tests in the earlier pass), is roughly 250 lines of recursive tree-building logic that the team would otherwise have to maintain, debug, and re-verify forever. Deleting it in favor of a well-known library reduces the project's long-term maintenance burden significantly.
- **Same interface, so nothing upstream had to change:** both estimators still expose fit(X, y) / predict(X) (Random Forest) or fit(X) / score_samples(X) (Isolation Forest), so anomaly_detection_v2.py and rul_model_v2.py needed only their internals swapped - dashboard_v2.py, evaluation_v2.py, train.py, and predict_service.py needed no changes at all.

Honest Limitation of This Update

This sandbox cannot install or import scikit-learn (no outbound internet access to PyPI, confirmed by a failed pip install attempt), so unlike every previous version in this project, this code could not be executed, run against real data, or benchmarked here. Every file was syntax-checked with python -m py_compile (all pass) and the API calls were written carefully against scikit-learn's documented interface, but this is not a substitute for actually running train.py end-to-end. Please run it on the machine where scikit-learn is installed and treat any benchmark numbers from the earlier from-scratch version as no longer representative of this implementation's actual accuracy.